

1. Vue d'ensemble du langage C++

Informations sur le cours

- Auteur : Bjarne Stroustrup
- Créé : Fin des années 1980 (V2.0 1989, V2.1 1990, V3.0 1991)
- Enseignant : J.M. Perronne
- Statut : Langage standardisé (ISO 14882)
- Standards publiés : C++98, C++03, C++11, C++14, C++17, C++21
- C++ est perçu comme une évolution du C (M. Ritchie et B.W. Kernighan, début des années 1970)
- Le cours couvre les concepts de langage compilé vs langages interprétés/scriptés

Classification du langage

- C++ moderne : Langage compilé (pas un langage de script)
- Applications : Jeux, finance, programmation système
- Processus de compilation : Code source → Code objet (compréhension machine) → Code exécutable (liaison, relocalisation d'adresse)

2. Structure du programme et modules

Anatomie d'un programme C++

- Programme principal : Point d'entrée de l'exécution main()
- Modules : Séparés en en-têtes (.h) et implémentation (.cpp)
- Composants du module :
 - En-tête (A.h) : Définition contractuelle de l'interface
 - Implémentation (A.cpp) : Code objet
 - Bibliothèque (L.h) : Interface contractuelle de la bibliothèque, code objet compilé

Processus de compilation et de liaison

1. Compilation : En-tête + Implémentation → Code objet
 2. Liaison : Combine les codes objets en exécutable
 3. Préprocesseur : S'exécute automatiquement avant la compilation
- Effectue des transformations de texte : inclusion de fichiers, suppression de texte, remplacement de texte
 - Les opérations suivent les directives lues dans le fichier analysé

3. Directives du préprocesseur

Définition Le préprocesseur est un programme qui analyse un fichier texte et effectue des transformations avant l'exécution de la compilation.

Commandes communes Toutes les commandes commencent par # au début d'une ligne

Inclusion de fichiers

```
#include "nomFichier"      // Inclusion normale de fichier
#include <nomFichier>        // Inclusion de bibliothèque standard
```

Définition de macro et remplacement de texte

```
#define aSymbol
#define First 1
```

Compilation conditionnelle

```
#ifdef identificateur
...
#endif
#ifndef (si pas défini...)
#elif (sinon si...)
#if (si condition...)
```

Directive Pragma

```
#pragma arguments
```

- Fournit des instructions spécifiques au compilateur tout en maintenant la portabilité du code
- Non standardisé entre tous les compilateurs

Constantes du préprocesseur

- **LINE** : Numéro de ligne actuel
- **FILE** : Nom du fichier actuel
- **DATE** : Date de traitement du fichier
- **TIME** : Heure de traitement du fichier
- **__cplusplus** : Identifiant du compilateur

Motif de garde d'en-tête Empêche l'inclusion double des en-têtes :

```
#ifndef TOTO_h
#define TOTO_h
// contenu de l'en-tête
#endif
```

Alternative (C++11) : `#pragma once` (même effet mais pas dans tous les compilateurs)

Exemple de macro Assert

- Utilisé pour vérifier les conditions à l'exécution
- Assertion sans code (si pas défini)
- Direction standard sur une ligne
- Peut être supprimé dynamiquement sans toucher le code

4. Types de données et variables

Types de données élémentaires

Type	Taille (octets)	Plage	Signification
char	1	-128 à 127	Caractère
unsigned char	1	0 à 255	Caractère non signé
short	2	-32768 à 32767	Entier court
int	2/4	-2147483648 à 2147483647	Entier
unsigned int	4	0 à 4 294 967 295	Entier non signé
long int	4	-2147483648 à 2147483647	Entier long
unsigned long int	4	0 à 4 294 967 295	Entier long non signé
float	4	-3.410^{-38} à 3.410^{38}	Flottant (précision simple)
double	4/8	-3.410^{-4932} à 3.410^{4932}	Double précision flottant
bool	varie	true/false	Booléen

Déclaration et portée des variables

- Portée : Entre accolades { ... } ou globale
- Points de déclaration : Pas d'emplacements prédéfinis
- Déclaration dans les expressions : Autorisée

```
int a;
int c, d;
for(int i=0; i < MAX; ++i) {...}
```

Initialisation des variables

- Avec des valeurs connues : `int a = 1;`
- Avec des valeurs de variable : `int b = a;`
- Initialisation uniforme (C++11) :
 - `int c{};` → s'initialise à 0
 - `int d{5};` → s'initialise à 5
- Note : N'accepte pas les conversions dégradantes

Inférence de type (C++11) auto Keyword

- Le type de variable est déterminé par l'initialiseur
- Doit être initialisé à la déclaration
- À utiliser seulement si nécessaire (peut nuire à la lisibilité et à la maintenabilité)
- Justifié pour l'utilisation de templates

```
auto varInt = 12; // varInt est un entier
```

decltype Keyword

- Déclare une variable avec un type identique à une autre

```
decltype(varInt) anOtherVarInt = 25;
```

Création de nouveaux types typedef

```
typedef type_original nouveau_type;  
typedef int Integer;
```

using (C++11)

```
using nouveau_type = type_original;  
using Integer = int;
```

Conversion de type Conversion implicite

```
int i = 2.5; // Perte de précision
```

Conversion explicite (Cast)

```
int i = (int) 2.5;
```

Constantes Méthodes de définition

```
#define CTE 1 // Définition de macro  
const int MAX = 10; // Variable constante  
const float CTEF = 1.0; // Constante flottante  
int tabInt[MAX]; // Valide en C++
```

const vs #define

- const fournit la sécurité de type et une portée appropriée
- #define est un remplacement de texte par le préprocesseur
- Préférer const pour les constantes sûres de type

5. Opérateurs

Catégories d'opérateurs Opérateurs arithmétiques +, -, *, /, %

Opérateurs d'incrément/décément ++, -- (préfixe et postfixe)

Opérateurs de comparaison ==, !=, >, <, >=, <=

Opérateurs logiques || (OU), && (ET), ! (NON)

Opérateurs bit à bit &, |, ^, >>, <<

Opérateurs d'affectation +=, -=, *=, /=, %=, <<=, >>=, &=, ^=, |=

Opérateur ternaire (condition) ? résultat_si_vrai : résultat_si_faux

Opérateur de tableau []

Opérateur de cast ()

Évaluation court-circuit

- Tous les opérandes n'ont pas besoin d'évaluation
- (Exp Test) ? résultat_si_vrai : résultat_si_faux
- if (f || 28) && 8211 - si f est faux, 8211 n'est pas évalué
- Utile pour les expressions conditionnelles : int min = (a > b) ? b : a;

6. Structures de contrôle

Boucle for moderne (C++11) Basé sur l'itération d'intervalle avec for basé sur la plage :

```
int tabInt[] = {2,4,6,8,10};
for(int &v:tabInt)    std::cout << v << ' ';
```

- Utilisation de la référence &v permet la modification des éléments
- Syntaxe plus propre que les boucles for traditionnelles

7. Fonctions

Déclaration et définition de fonction Déclaration (En-tête)

```
type_de_donnee Nom_De_La_Fonction(type1 argument1, type2 argument2, ...) {
    // Corps de la fonction
}
```

Exemples

```
void f();  
float pi();  
int max(int, int); // Fonction avec deux arguments
```

Appels de fonction

```
Nom_De_La_Fonction(); // Aucun argument  
Nom_De_La_Fonction(argument1, argument2); // Avec arguments  
int a = Nom_De_La_Fonction(); // Stockage de la valeur de  
retour
```

Arguments par défaut

```
void printPrompt(char* p = "login > ") {  
    cout << endl << p;  
}
```

- La fonction peut être appelée avec ou sans arguments
- Les défauts sont utilisés lorsque les arguments ne sont pas fournis

Surcharge de fonction C++ permet plusieurs fonctions avec le même nom mais des paramètres différents :

```
void whoAmI(int i) { cout << "Je suis l'entier :" << i << endl; }  
void whoAmI(char c) { cout << "Je suis le char :" << c << endl; }
```

- Différenciation basée sur les types/quantités d'arguments
- Le compilateur sélectionne la version appropriée basée sur l'appel

Pointeurs de fonction Un pointeur contient l'adresse du début du code binaire de la fonction : Déclaration

```
type (*identificateur)(type arg1, type arg2, ...);  
int (*ptrFct) (int, int); // Pointeur vers fonction retournant int avec  
deux paramètres int
```

Utilisation de typedef

```
typedef int (*PtrFunc) (int, int);
```

Utilisation des pointeurs de fonction

```
PtrFunc tabFuncs[]={subb, add, max};
for(int i=0; i<=sizeof(tabFuncs)/sizeof(PtrFunc); ++i)    cout << endl <<
do(tabFuncs[i],2,3);
```

8. Opérations d'entrée/sortie

Sortie vers le flux standard cout et opérateur <<

```
#include <iostream>
using namespace std;
cout << "hello ";
int a = 1;
cout << a;
cout << endl << "a " << a;
```

Types multiples

```
int n = 25;
char c = 'a';
double x = 3.15;
char *mess = "hello";
cout << "valeur de n : " << n << endl;
cout << "valeur de c : " << c << endl;
cout << "mess : " << mess << endl;
cout << "adresse de n : " << &n;
```

Entrée depuis le flux standard cin et opérateur >>

```
#include <iostream>
using namespace std;
int a;
cin >> a;
cout << endl << "a :" << a;
```

Lecture de chaînes de caractères

```
char string[80];
for (unsigned int n=0; n<79; n++) {
    cin >> string[n];
    if (string[n]=='.') {
        string[n] = '\\0';
        break;
    }
}
```

```

    }
}
cout << "entrées : " << string;

```

9. Références et pointeurs

Références Définition

- Alternative aux pointeurs
- Représente une adresse (comme un pointeur)
- La manipulation agit sur l'objet référencé (pas sur la référence elle-même)
- Doit être initialisée à la déclaration
- Peut être perçue comme un pointeur constant avec déréférencement automatique
- L'adresse ne peut pas être changée après l'initialisation
- Utilisée comme synonymes pour les variables

Utilisation des références

```

int a = 1;
int &aref = a;
cout << aref << endl; // Sortie : 1
aref = 2;
cout << a << endl;    // Sortie : 2

```

Pointeurs Définition Un pointeur est une variable dont la valeur est an address. Les pointeurs sont typés.

Exemples de déclaration

```

int *a;           // Pointeur vers entier
int b = 12;
a = &b;           // a prend la valeur de l'adresse de b
cout << a;        // Afficher l'adresse : 0xbffffa7cc
cout << *a;       // Afficher le contenu (valeur de b) : 12

```

Arithmétique des pointeurs

```

int tab[5]={1,3,5,7,11};
int *ptr = tab; // Pointeur prend la valeur de tab
for(int i=0; i<5; ++i, ptr++) cout << ' ' << *ptr; // 1 3 5 7 11

```

Initialisation des pointeurs (C++11)

- Avant : `int *ptr_old = NULL;`
- Maintenant : `int *ptr_new = nullptr; (type std::nullptr_t)`

Initialisation moderne des pointeurs

- nullptr est préféré à NULL en C++11+

Tableaux comme pointeurs "Un tableau est également un pointeur. L'arithmétique des pointeurs s'applique aux tableaux"

- Les tableaux sont aussi des pointeurs
- L'arithmétique des pointeurs s'applique aux tableaux

Passage d'arguments aux fonctions Par pointeur

```
void swap(int * a, int * b) {  
    int tmp = *a;  
    *a = *b;  
    *b = tmp;  
}  
int a=1, b=2;  
swap(&a, &b);  
cout << "a " << a << " b " << b; // Sortie : a 2 b 1
```

Par référence (Préféré)

```
void swap(int & a, int & b) {  
    int tmp = a;  
    a = b;  
    b = tmp;  
}  
int a=1, b=2;  
swap(a, b);  
cout << "a " << a << " b " << b; // Sortie : a 2 b 1
```

"On ne passe jamais en paramètre un objet par valeur !!"

- Ne jamais passer des objets par valeur en paramètres
- Peut déborder la pile d'appel
- Référence préférée au pointeur
- Dans 9 cas sur 10, passer par référence

10. Chaînes de caractères

Définition de chaîne en C++ Une chaîne de caractères C++ est un tableau de caractères ou un tampon terminé par le caractère nul '\0' :

Déclaration de pointeur

```
char *mess = "hello";  
// Représentation : 'h' 'e' 'l' 'l' 'o' '\0'
```

Déclaration de tableau

```
char mess[] = {'h','e','l','l','o','\0'};
```

Caractères spéciaux

Caractère	Signification
\0	Caractère nul (terminateur de chaîne)
"	Guillemet double
\	Barre oblique inverse
\a	Alerte/cloche
\b	Retour arrière
\f	Saut de page
\n	Nouvelle ligne
\t	Tabulation
\v	Tabulation verticale

Bibliothèque de chaînes La bibliothèque fournit des fonctions pour la manipulation de chaînes de caractères :

- Copie de chaîne
- Comparaison de chaîne
- Concaténation de chaîne
- Et plus...

11. Types de données complexes

Tableaux Déclaration

```
type Nom_du_tableau [Nombre d'éléments]
int tabInt1[5]; // Tableau de 5 entiers
int tabInt2[2] = {12, 5}; // Tableau initialisé à la
déclaration
int matrix[2][2] = {{12, 5},{3,8}}; // Tableau 2D
```

Itération

```
for(int i=0; i<2; i++) cout << tabInt2[i];
// Sortie : 12 5
```

Structures But Créer un type composite où les champs n'ont pas le même type.

Déclaration

```
struct NomStructure {  
    typeChamp1 champs1;  
    typeChamp2 champs2;  
    typeChampN champsN;  
};
```

Exemple

```
struct Point {  
    int x;  
    int y;  
};  
struct ColoredPoint {  
    Point point;  
    int color;  
};
```

Utilisation des structures

```
Point p1;  
Point *p2 = &p1;  
p1.x = 10;  
p1.y = 20;  
ColoredPoint cp;  
cp.point.x = 10;  
cp.color = 0;
```

Opérateurs

- Opérateur point (.) : Accès direct aux membres
- Opérateur flèche (->) : Accès aux membres via pointeur

```
Point *p2 = &p1;  
cout << p2->x // Équivalent à (*p2).x
```

Types énumérés Définition Les types énumérés sont des types entiers (basés sur des entiers). Chaque valeur d'énumération a un nom unique.

Déclaration

```
enum NomType {v1, v2, ..., vn};
enum NomType {v1 = 5, v2, ..., vn};    // Avec initialisation
enum NomType {v1 = 5, v2 = 7, ...};    // Initialisation individuelle
```

Exemple

```
enum Position {top, bottom, left, right};
Position pos = left;
switch (pos) {
    case top: cout << "top"; break;
    case bottom: cout << "bottom"; break;
    case left: cout << "left"; break;
    case right: cout << "right"; break;
}
```

Énumérations fortement typées (C++11) Motivations

- Pas de conversions implicites vers entier
- Arithmétique impossible
- Possibilité de choisir la taille de l'entier sous-jacent

Syntaxe

```
enum class Direction : short { up, down, left, right };
Direction dir = Direction::up;
switch (dir) {
    case Direction::up : cout << "up"; break;
    case Direction::down : cout << "down"; break;
    case Direction::left : cout << "left"; break;
    case Direction::right : cout << "right"; break;
}
```

12. Gestion de la mémoire dynamique

Allocation et désallocation de mémoire Opérateur new (Allocation)

```
new Type;           // Allocation d'objet unique
new Type[n];        // Allocation de tableau
```

Opérateur delete (Désallocation)

```
delete adresse;     // Désallocation d'objet unique
delete[] adresse;   // Désallocation de tableau
```

Exemple

```
const int N = 10;
int *ptr, *tab;
ptr = new int;
tab = new int[N];
delete ptr;
delete[] tab;
```

Placement new Deux opérateurs de placement prédéfinis qui retournent l'adresse passée en paramètre (n'utilisez pas delete) :

```
new(adresse) type;
new(adresse) type[n];
```

13. Programmation orientée objet

Classes et objets Définition

- Classe : Type, ensemble d'objets avec structure similaire
- Objet : Instance d'une classe, entité spatio-temporelle

Composants de classe

- Identité : Différencie l'objet dans la classe
- État : Attributs (membres de données)
- Comportement : Méthodes (fonctions membres)

Structure de déclaration de classe Fichier d'en-tête (.h)

```
#ifndef UN_SYMBOLE_UNIQUE
#define UN_SYMBOLE_UNIQUE
class NomClasse: public|protected|private NomClasseMere,... {
    public|protected|private:
        // Déclaration de membres de données et/ou méthodes
    public|protected|private:
        // Déclaration de membres de données et/ou méthodes
};
#endif
```

Fichier d'implémentation (.cpp)

```
#include "nomfichier.h"
// Implémentation des méthodes de classe
```

```
// Déclarations spécifiques
```

Modificateurs d'accès Règles de visibilité

Type d'accès	public	protected	private
héritage public	public	protected	inaccessible
héritage protected	protected	protected	inaccessible
héritage private	private	private	inaccessible

Mots-clés

- public : Accès pour tout le monde
- protected : Accès pour les classes de la même famille
- private : Accès limité à la classe elle-même

Encapsulation

- Support du principe d'encapsulation
- Accès aux entités défini par les mots-clés
- L'héritage module la visibilité des composants ancêtres

Exemple : Classe Point En-tête (point.h)

```
#ifndef POINT_H
#define POINT_H
class Point {
public:
    Point();
    Point(int, int);
    int getX() const;
    int getY() const;
    void setX(int);
    void setY(int);
private:
    int x, y;
};
#endif
```

Implémentation (point.cpp)

```
#include "point.h"
Point::Point(): x(0), y(0) { }
Point::Point(int _x, int _y): x(_x), y(_y) { }
int Point::getX() const { return x; }
int Point::getY() const { return y; }
void Point::setX(int _x) { x = _x; }
void Point::setY(int _y) { y = _y; }
```

Utilisation (demo.cpp)

```
int main () {
    Point p1, p2(10,20);
    Point p3(p2);
    cout << "p1:" << p1.getX() << ',' << p1.getY();
    cout << "p2:" << p2.getX() << ',' << p2.getY();
    cout << "p3:" << p3.getX() << ',' << p3.getY();
    return 0;
}
// Sortie : p1:0,0  p2:10,20  p3:10,20
```

Constructeurs Trois types

1. Constructeur par défaut : Aucun paramètre
2. Constructeur valorisé : Avec paramètres
3. Constructeur de copie : Prend une référence à un objet de la même classe

Règles d'existence

- Le constructeur par défaut est généré tant qu'aucun constructeur n'est explicitement défini
- Le constructeur de copie est généré s'il n'est pas redéfini
- Les constructeurs s'invoquent automatiquement lors de l'instanciation d'objet

Règles de comportement

- Les constructeurs vides (sauf copie) appellent le constructeur par défaut de la classe parente
- Initialisent les membres de données en invoquant leurs constructeurs par défaut
- Le constructeur de copie appelle le constructeur de copie de la classe parente
- Par défaut, initialise les membres de données "membre à membre"

Exemple avec les trois

```
class Point {
public:
    Point();                // Défaut
    Point(int, int);        // Valorisé
    Point(const Point&);     // Copie
    ~Point();
};
Point::Point(): x(0), y(0) { cout << "-> défaut"; }
Point::Point(int _x, int _y): x(_x), y(_y) { cout << "-> valorisé"; }
Point::Point(const Point& p): x(p.x), y(p.y) { cout << "-> copie"; }
Point::~~Point() { cout << "-> destructeur"; }
```

Destructeurs Définition

- Fonction membre appelée lorsque l'objet est détruit
- Même nom que la classe, précédé de ~
- Aucun argument, aucune valeur de retour
- Permet de libérer les ressources acquises

Règles d'existence et de comportement

- Généré tant qu'il n'est pas redéfini
- Invoqué automatiquement lors de la destruction d'objet
- Comportement par défaut : invoque les destructeurs des membres de données et du destructeur de la classe parente

Utilisation avec allocation dynamique

```
int main () {
    Point *p4 = new Point[3];
    Point *p5 = new Point(1,2);
    delete[] p4;
    delete p5;
    return 0;
}
```

Membres constants Membres de données constants Définis avec le mot-clé const :

```
class Truc {
public:
    Truc();
private:
    const int N;
};
Truc::Truc(): N(10) { }
```

- Initialisés lors de la construction
- Ne peuvent pas être changés après l'initialisation

Fonctions membres constantes Fonctions qui ne doivent pas modifier l'état de l'objet :

```
class Point {
public:
    int getX() const;
    int getY() const;
};
```

- Utilisées pour les fonctions de consultation (Méthodes Get)
- Exemple : getX() et getY() de la classe Point

Membres mutables Concept La constance peut être physique ou logique :

- L'objet apparaît constant aux clients
- Peut changer l'état interne pour assurer le comportement

Exemple

```
class Point {
private:
    int x, y;
    mutable int access;
public:
    int getX() const {
        access++;
        return x;
    }
};
```

- Sans mutable, le compilateur interdirait la modification de access dans la fonction const
- Permet les changements d'état interne dans les fonctions const

Membres et méthodes statiques Définition

- Membres statiques : Partagés entre toutes les instances
- Méthodes statiques : Appelables sans création d'instance
- Existent dès que la classe existe (pas dépendant des objets)
- Chaque objet possède des instances des membres ; les membres statiques sont partagés

Exemple : Compteur d'objets

```
class Class {
public:
    Class();
    Class(const Class&);
    virtual ~Class();
    static unsigned int getRef();
private:
    static unsigned int ref;
};

unsigned int Class::ref = 0;
Class::Class() { ref++; }
Class::Class(const Class& c) { ref++; }
Class::~~Class() { ref--; }
unsigned int Class::getRef() { return ref; }
```

Utilisation avec héritage

```

class Point: public Class { ... };
int main () {
    Point *p4 = new Point[3];
    // Crée 3 objets Point
    Point *p5 = new Point(1,2); // Crée 1 objet Point
    Point p6(*p5);              // Constructeur de copie
    cout << Class::getRef();    // Sortie : 5
    delete[] p4;
    delete p5;
    cout << Class::getRef();    // Sortie : 1
    return 0;
}

```

Affectation d'objet (Affectation) Comportement par défaut

- L'opérateur d'affectation par défaut existe
- L'affectation est membre à membre
- L'opérateur est redéfinissable

Exemple de redéfinition

```

class Point: public Class {
public:
    Point& operator = (const Point&);
};
Point& Point::operator = (const Point& p) {
    if(&p != this) {
        x = p.x;
        y = p.y;
    }
    return *this;
}

```

Note importante "Les redéfinitions du constructeur par recopie et de l'opérateur d'affectation vont de pair."

- Les redéfinitions du constructeur de copie et de l'opérateur d'affectation vont de pair
- Les deux devraient être redéfinis s'ils gèrent la mémoire dynamique

Conversion de type Conversion implicite via constructeur

```

Point(int x, int y = 0); // Définition du constructeur
Point p = 2;             // Conversion implicite
cout << "p:" << p.getX() << ',' << p.getY(); // Sortie : p:2,0

```

Constructeur explicite

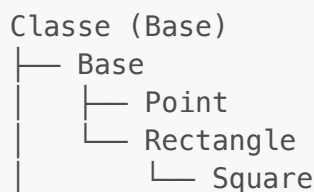
```
explicit Point(int x, int y = 0);
Point p = 2;           // Illégal
Point p = (Point) 2;  // OK
```

Opérateur de conversion

```
operator int ();
Point::operator int() {
    return sqrt(x*x + y*y);
}
Point p(2,2);
int i = p;
cout << i;  // Sortie : 2
```

14. Héritage et polymorphisme

Bases de l'héritage Hiérarchie de classe exemple



Définition de l'héritage

- Classe dérivée : Hérite des descriptions de la classe parente
- Héritage multiple : Interdit

Méthodes virtuelles et polymorphisme Fonctions virtuelles

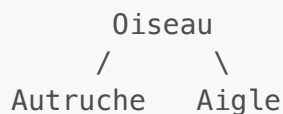
- Permettent aux classes dérivées de remplacer les méthodes de la classe de base
- Virtuel pur : Méthode abstraite (doit être remplacée)
- Polymorphisme : Capacité à avoir un comportement différent basé sur le type d'objet

Exemple de classe de base

```
class Base : public Class {
public :
    virtual std::ostream& printOn(std::ostream&) const = 0;
    friend std::ostream& operator << (std::ostream&, const Base&);
};
std::ostream& operator << (std::ostream& os, const Base& o) {
    return o.printOn(os);
}
```

- Virtuelle pure : Méthode abstraite (printOn)
- Classe Base est abstraite (a une méthode pure virtuelle)

Polymorphisme Exemple : Oiseaux



Comportement :

- Autruche : pas de vole (pas de vol)
- Aigle : vole (vole)

```

Oiseau * o1 = new Autruche();
Oiseau * o2 = new Aigle();
o1->vole(); // Dépend si virtuel
o2->vole();
  
```

Types de liaison :

- Liaison dynamique (si virtuel) : Appelle la méthode de la classe dérivée
- Liaison statique (si pas virtuel) : Appelle la méthode de la classe de base

Gestion de la mémoire en héritage

```

delete o1; // Appelle le destructeur (les destructeurs ne sont
           généralement pas remplacés)
  
```

- Utiliser virtuel dans le destructeur pour assurer un nettoyage approprié
- Empêche les fuites de mémoire

Implémentation de classe dérivée Classe Point héritant de Base

```

class Point: public Base {
public:
    Point();
    Point(int, int);
    Point(const Point&);
    virtual ~Point();
    virtual Point& operator = (const Point&);
    virtual int getX() const;
    virtual int getY() const;
    virtual void setX(int);
    virtual void setY(int);
    virtual void translate(int, int);
  
```

```

        virtual std::ostream& printOn(std::ostream&) const;
private:
    int x, y;
};

```

Classe Rectangle

```

class Rectangle: public Base {
public:
    Rectangle();
    Rectangle(const Point&, const Point&);
    Rectangle(int, int, int, int);
    virtual Point getTopLeft() const;
    virtual Point getBottomRight() const;
    virtual void setTopLeft(const Point&);
    virtual void setBottomRight(const Point&);
    virtual void translate(int dx, int dy);
protected:
    virtual std::ostream& printOn(std::ostream&) const;
private:
    Point topLeft, bottomRight;
};

```

Classe Square avec héritage privé

```

class Square: private Rectangle {
public:
    Square() {};
    Square(const Point&, int);
    Square(int, int, int);
    virtual Point getLocation() const;
    virtual int getSize() const;
    virtual void setLocation(const Point&);
    virtual void setSize(int);
    virtual void Translate(int, int);
    friend std::ostream& operator << (std::ostream&, const Square&);
};

```

Héritage privé Note :

- Square "est-un" Rectangle en interne
- Le contrat Rectangle est caché et inaccessible
- Le contrat Square est complètement redéfini
- L'interface publique est différente de Rectangle

Délégation de constructeur par défaut (C++11)

```
Point::Point(): Point(0,0) {}
```

- Nouvelle fonctionnalité permettant aux constructeurs d'appeler d'autres constructeurs
- Réduit la duplication de code

Gestion d'exception dans les objets Cas pertinents

- Constructeur : Exception lors de la création d'objet
- Destructeur : Ne devrait pas lancer (une fois instancié)
- Erreurs d'entrée/sortie : Ne peuvent pas être empêchées
- Erreurs de programmation : Peuvent être corrigées pendant le développement
- Assertions : Si désactivées, le code de gestion d'erreur peut être supprimé

15. Templates et programmation générique

Templates de fonction But Générer du code de fonction pour différents types (même fonction, résultats différents) :

- Nom de type comme paramètre générique
- Types générés basés sur l'entrée de l'appel de fonction

Syntaxe

```
template<class T>
void swap(T& a, T& b) {
    T tmp = a;
    a = b;
    b = tmp;
}
template<class T>
T& max(T& a, T& b) {
    return (a > b) ? a : b;
}
template<class T>
void test(T& x, T& y) {
    cout << "x: " << x << " y: " << y;
    swap(x, y);
    cout << "x: " << x << " y: " << y;
    cout << "max(x,y): " << max(x, y);
}
```

Utilisation

```
int x = 1, y = 10;
test(x, y);
string s1 = "s1", s2 = "s2";
test(s1, s2);
// Sortie pour entiers :
```

```
// x: 1 y: 10
// x: 10 y: 1
// max(x,y): 10
```

Templates de classe Exemple de tableau dynamique

```
template <class T>
class Array: public Base {
public:
    typedef unsigned int size_a;
    Array();
    Array(size_a);
    virtual ~Array();
    virtual T& operator [] (size_a);
    virtual ostream& printOn(ostream &) const;
private:
    T *array;
    size_a size;
};

template <class T>
Array<T>::Array(): size(1), array(new T[1]) { }

template <class T>
Array<T>::Array(size_a sz): size(sz), array(new T[sz]) { }

template <class T>
Array<T>::~~Array() { delete[] array; }

template <class T>
T& Array<T>::operator [] (size_a i) {
    if (i > (size-1))
        throw "invalid index";
    return array[i];
}

template <class T>
ostream& Array<T>::printOn(ostream& s) const {
    for (size_a i=0; i<size; ++i)
        s << array[i] << ' ';
    return s;
}
```

Utilisation

```
try {
    Array<int> aint(3);
    aint[0] = 1;
    aint[1] = 2;
    aint[2] = 3;

    Array<string> astring(2);
    astring[0] = "st1";
    astring[1] = "st2";
}
```

```
    cout << aint << endl;
    cout << astring << endl;
    astring[3] = "st3"; // Lance "invalid index"
} catch(const char* msg) {
    cout << endl << msg;
}
```

Lignes directrices pour les templates Notes importantes

- Templates dans les en-têtes : Les templates doivent être compilés avec chaque utilisation
- typename vs class : Les deux peuvent être utilisés de manière interchangeable
- Exigences de type : T doit supporter les opérations utilisées dans le template (par exemple, comparaison)
- Commenter sur l'élément : Documenter ce que le type T doit supporter

16. Gestion d'exception

Mécanisme d'interception d'erreur Gère :

- Signal remontée vers les fonctions appelantes
- Détection du signal d'erreur par la fonction réceptive
- Exécution du traitement approprié

Implémentation en deux phases Phase 1 : Bloc réceptif

```
try {
    // Instructions susceptibles d'erreurs
}
```

Phase 2 : Gestionnaires d'erreurs

```
catch (TypeException1 e1) {
    // Instructions pour gérer l'exception
}
catch (TypeException2 e2) {
    // Instructions pour gérer l'exception
}
catch(...) {
    // Exécuté si aucun autre catch ne correspond
}
```

Déclenchement d'erreur

```
throw exception_object;
```


Hiérarchie de classe d'exception Exception de base

```
class exception {
public:
    exception() throw();
    exception(const exception &) throw();
    exception &operator=(const exception &) throw();
    virtual ~exception() throw();
    virtual const char *what() const throw();
};
```

Types d'exception Erreurs logiques (Erreurs de programmation) :

- domain_error
- invalid_argument
- length_error
- out_of_range

Erreurs d'exécution (Entrée/sortie) :

- range_error
- overflow_error
- underflow_error

Exemple d'exception personnalisée

```
class IllegalMonthException : public Base {
public:
    IllegalMonthException(unsigned char);
    virtual std::ostream& printOn(std::ostream&) const;
private:
    const unsigned short illegalMonth;
};
IllegalMonthException::IllegalMonthException(unsigned char m):
    illegalMonth(m) {}
std::ostream& IllegalMonthException::printOn(std::ostream& os) const {
    return os << "mois illégal : " << illegalMonth;
}
```

Utilisation

```
const char* Monthconverter::convert(unsigned short m) {
    if(m < 1 || m > 12)
        throw IllegalMonthException(m);
    return month[m];
}
int main () {
    unsigned short month;
```

```
while(true) {
    try {
        cout << "mois à convertir : ";
        cin >> month;
        cout << "-> " << Monthconverter::convert(month);
    } catch(const IllegalMonthException& e) {
        cout << endl << e;
    }
}
return 0;
}
```

17. Bibliothèque standard

Catégories Support du langage

- Exceptions de base : exception, bad_exception, bad_alloc
- Exceptions complémentaires : domain_error, invalid_argument, logic_error, length_error, out_of_range, range_error, runtime_error, overflow_error, underflow_error
- Identification de type dynamique

Utilitaires généraux

- Générateurs d'opérateurs
- Structure pair
- Classes de fonctions
- Classe String

Conteneurs Conteneurs de séquence :

- vector : Mémoire continue, accès aléatoire immédiat
- list : Mémoire séquentielle, pas d'accès aléatoire
- deque : File d'attente double extrémité

Types abstraits :

- queue
- priority_queue
- stack

Conteneurs associatifs :

- set : Valeurs uniques
- multiset : Valeurs dupliquées autorisées
- map : Paires clé/valeur (dictionnaire)
- multimap : Valeurs multiples par clé

Itérateurs

- Itèrent uniformément sur les conteneurs
- Les mêmes algorithmes s'appliquent à tous les conteneurs

Algorithmes

- Composants effectuant le traitement depuis/vers les conteneurs

Numérique

- complex
- valarray

Entrée/Sortie

- ifstream : Flux d'entrée fichier
- istream : Flux d'entrée
- istream : Flux d'entrée chaîne
- ofstream : Flux de sortie fichier
- ostream : Flux de sortie
- ostream : Flux de sortie chaîne
- ios_base : Classe de base de flux I/O
- basic_ios : Classe de base I/O de base

18. Opérations d'entrée/sortie (Avancé)

Manipulation du format de flux Fichier à inclure

```
#include <iomanip>
```

Modificateurs de base

- dec, hex, oct : Modifier la base pour l'entrée/sortie entière
- showbase, noshowbase : Afficher/masquer l'indicateur de base

Notation flottante

- fixed, scientific : Notation fixe ou scientifique pour les flottants
- showpoint, noshowpoint : Générer le point décimal inconditionnellement

Alignement

- left, right : Alignement gauche ou droite

Affichage du signe

- showpos, noshowpos : Afficher/masquer le signe plus

Casse

- uppercase, nouppercase : Lettres majuscules/minuscules pour hex

Espace blanc

- skipws, noskipws : Ignorer/respecter l'espace blanc en entrée

Largeur et précision

- `setw(int)` : Définir la largeur de champ
- `setfill(int)` : Définir le caractère de remplissage
- `setprecision(int)` : Définir les décimales

Réinitialisation/définition d'indicateur

- `resetiosflags(ios_base::fmtflags)` : Réinitialiser des indicateurs spécifiques
- `setiosflags(ios_base::fmtflags)` : Définir des indicateurs spécifiques

Exemples

```
cout << "15 décimal-> " << 15;           // 15 décimal-> 15
cout << hex << "15 hexa-> " << 15;         // 15 hexa-> f
cout << showbase << hex << "15 hexa avec base-> " << 15; // 15 hexa avec
base-> 0xf
cout << fixed << "notation fixe-> " << 10.234; // notation fixe->
10.234000
cout << scientific << "scientifique-> " << 10.234; // scientifique->
1.023400e+001
cout << setfill('-') << setw(15) << 10.234; // -----10.234000
cout << setprecision(2) << "précision 2-> " << 10.234; // précision 2->
10.23
```

Gestion de fichiers Fichier à inclure

```
#include <fstream>
```

Copie de fichier simple

```
ifstream fin("demo.txt");
ofstream fout("demo-copy.txt");
fin >> noskipws;
char c;
while (fin) {
    fin >> c;
    fout << c;
}
```

Opérations de flux de chaîne Fichier à inclure

```
#include <sstream>
```

Valeur vers chaîne

```
template<class T>
string value2string(const T& v) {
    ostringstream os;
    os << v;
    return os.str();
}
```

Chaîne vers valeur

```
template <class T>
T string2value(const string& s) {
    istringstream is(s);
    T v;
    is >> v;
    return v;
}
```

Utilisation

```
string s("result: ");
cout << s + value2string(string2value<int>("2") + 3); // Sortie : result:
5
```

19. Expressions lambda (C++11)

Fonctions anonymes et fermetures But

- Fonctions sans noms (expressions lambda)
- Fermetures capturent des variables de la portée externe
- Doivent spécifier les variables capturées

Syntaxe de base

```
typedef std::function<int (int ,int)> PtrFunc;
PtrFunc max = [](int left ,int right)->int {
    return (left >= right) ? left : right;
};
PtrFunc add = [](int left ,int right)->int {
    return left + right;
};
std::cout << std::endl << max(12, 5); // Sortie : 12
```

Fermetures avec capture de variable

```
int x = 100, y = 200;
auto do_fermeture = [x, y](PtrFunc f)->int {
    return f(x, y);
};
PtrFunc tabFuncs[] = {max, add, subs};
for(PtrFunc &f : tabFuncs)
    std::cout << std::endl << do_fermeture(f);
// Sortie : 200 300 -100
```

Modes de capture

- [] : Aucune variable externe
- [x, &y] : x par valeur, y par référence
- [&] : Toutes les variables par référence
- [=] : Toutes les variables par valeur
- [&, x] : x par valeur, autres par référence
- [=, &x] : x par référence, autres par valeur

20. Conteneurs et itérateurs

Vector et List vector

- Mémoire : Continue
- Accès : Immédiat (accès aléatoire)
- Opérations : Ajout/lecture en vrac rapide
- Vitesse : Optimisé pour l'accès séquentiel

list

- Mémoire : Séquentielle
- Accès : Accès aléatoire impossible
- Opérations : Ajout/lecture ponctuelle rapide
- Ordre : Maintient l'ordre d'insertion

Méthodes d'itération Itérateur traditionnel

```
vector<float> v1(tab, tab+5);
vector<float>::iterator it = v1.begin();
for(; it != v1.end(); ++it)    cout << *it << " ";
```

For-each avec lambda (C++11)

```
auto print = [](float f) { std::cout << f << ', '; };
std::vector<float> v1 = {1,2,3,4,5};
for(float v : v1)    std::cout << v << " ";
```

Algorithme for_each

```
void print(float f) { cout << f << ','; }
for_each(v1.begin(), v1.end(), print);
// Ou avec lambda
for_each(v1.begin(), v1.end(), [](float f) { std::cout << f << ','; });
```

Itérateurs de flux de sortie

```
ostream_iterator<float> outPut(cout, "-");
copy(v1.begin(), v1.end(), outPut);
// Sortie : 1-2-3-4-5-
```

Itérateur de flux d'entrée

```
vector<float> v2;
istream_iterator<float> inIter(cin), endIter;
copy(inIter, endIter, back_inserter(v2));
```

Tableaux de taille fixe (C++11) But Utiliser des tableaux comme conteneurs STL

Syntaxe

```
#include <array>
std::array<float, 5> a1 = {1,2,3,4,5};
std::cout << "taille : " << a1.size();
(a1.empty()) ? std::cout << "a1 vide " : std::cout << "a1 pas vide ";
std::cout << "front() : " << a1.front();
std::cout << "back() : " << a1.back();
std::cout << "at() avec limites : " << a1.at(1);
std::cout << "[] : " << a1[1];
copy(begin(a1), end(a1), std::ostream_iterator<float>(std::cout, "-"));
```

21. Techniques avancées

Évaluation de fonction et courbes discrètes Template Evaluator

```
template <class T>
class Evaluator {
public:
    class EvalFunc {
    public:
        virtual T operator () (const T&) = 0;
    };
};
```

```

typedef pair<vector<T>, vector<T>> Shape;

static Shape buildShape(const T& min, const T& max, const T& step,
EvalFunc&);
static ostream& printShape(ostream&, const Shape& s);
};
template <class T>
typename Evaluator<T>::Shape Evaluator<T>::buildShape(
const T& min, const T& max, const T& step, EvalFunc& f) {
vector<T> x, y;
for (T i = min; i <= max; i += step) {
y.push_back(f(i));
x.push_back(i);
}
return Shape(x, y);
}

```

Exemple de fonction triangle

```

template<class T>
class triangleFunc : public Evaluator<T>::EvalFunc {
public:
triangleFunc(const T& _min, const T& _mid, const T& _max) :
min(_min), mid(_mid), max(_max) {}

virtual T operator() (const T& x) {
if (x < min || x > max)
return 0;
return (x <= mid) ?
(x - min) / (mid - min) :
(max - x) / (max - mid);
}
private:
T min, mid, max;
};

void testShape() {
triangleFunc<float> func(2.5f, 5, 7.5f);
Evaluator<float>::Shape s = Evaluator<float>::buildShape(0, 10, 0.5f,
func);
Evaluator<float>::printShape(cout, s);
}

```

Comptage de lignes

```

#include <iostream>
#include <fstream>
#include <algorithm>
#include <iterator>

```



```

int main (int argc, char * const argv[]) {
    if (argc < 2) {
        cout << "nom de fichier manquant";
        return 0;
    }

    ifstream file(argv[1]);
    istream_iterator<char> it(file), itEnd;

    if (file) {
        file >> noskipws;
        cout << "nombre de lignes : " << count(it, itEnd, '\n');
    }
    return 0;
}

```

Compteur de mots

```

class WordCounter {
public:
    WordCounter() = default;
    WordCounter(std::istream& s);
    void count(std::istream& s);
    std::ostream& printOn(std::ostream& os) const;
private:
    typedef std::map<std::string, unsigned long> w_map;
    typedef w_map::value_type w_value;
    typedef w_map::iterator iterator;
    typedef w_map::const_iterator const_iterator;
    w_map words;
};

WordCounter::WordCounter(std::istream& s) { count(s); }
void WordCounter::count(std::istream& s) {
    std::string current;
    while(s) {
        s >> current;
        (*words.insert(w_value(current, 0)).first).second++;
    }
}

std::ostream& WordCounter::printOn(std::ostream& os) const {
    for(const_iterator it = words.begin(); it != words.end(); ++it)
        os << (*it).first << "->" << (*it).second << std::endl;
    return os;
}

```

22. Exercices pratiques et devoirs

TD HelloWorld Premier HelloWorld Programme simple imprimant "Hello world" vers la sortie standard

Deuxième HelloWorld Programme utilisant un module pour imprimer :

```

HelloWorld
├── main() {...}
├── helloworld.h (ostream& printHello(ostream&))
└── helloworld.cpp (implémentation)

```

TD DynamicArray Exigences Créer une classe DynamicArray représentant un tableau dynamique d'entiers :

- Instanciation : Défaut, avec spécification de taille, ou par copie
- Information : Taille et taille de bloc d'allocation
- Accès : Méthodes Get/set et operator[]
- Croissance : Augmenter la taille à la demande
- Sortie : Représentation textuelle sur flux de sortie

Version générique Créer une classe template Array :

- Même fonctionnalité que DynamicArray
- Fonctionne avec tout type T
- Supporte le dimensionnement dynamique

TD Dictionnaire Diagramme de classe

```

Base
├── Dictionary
│   ├── - words: map<string, string>
│   ├── + Dictionary()
│   ├── + Dictionary(istream&)
│   ├── + Flush(): void
│   ├── + Size() const: unsigned int
│   ├── + Insert(const string&, const string&): void
│   ├── + Remove(const string&): void
│   ├── + Translate(const string&) const: string
│   ├── # ReadFrom(istream&): void
│   ├── # ReadPair(istream&): void
│   └── # PrintOn(ostream&) const: ostream&
└── Exception
    ├── - string: what
    ├── + Exception(const string&)
    ├── + What() const: string
    └── # PrintOn(ostream&) const: ostream&
        └── DictionaryException
            └── + DictionaryException(const string&)

```

Implémentation de la classe Dictionary

```

class Dictionary : public Base {
public:

```

```

Dictionary();
Dictionary(istream&);
virtual ~Dictionary() {}
virtual void flush();
virtual unsigned int Size() const;
virtual void insert(const string&, const string&);
virtual void remove(const string&);
virtual string translate(const string&) const;
protected:
    virtual ostream& printOn(ostream&) const;
    virtual istream& readFrom(istream&);
    virtual istream& readPair(istream&);
    friend istream& operator >> (istream&, Dictionary&);
private:
    typedef map<string, string> Words;
    typedef Words::value_type value_type;
    typedef Words::iterator iterator;
    typedef Words::const_iterator const_iterator;
    Words words;
};

```

23. C avec intégration C++

Utilisation de extern "C" But Inclure des en-têtes de bibliothèque C dans le code C++

Syntaxe

```

extern "C" {
    #include "alib.h"
    void f(); // Fonction C
}

```

Notes

- Permet d'utiliser des fonctions C dans des programmes C++
- Différent mangling de nom de fonction entre C et C++
- Doit être explicitement déclaré pour éviter les erreurs de liaison